

Autonomous Car — 12-Week Plan (rev. 2)

Revised 6 September 2026 | Reordered for **offline-first** development: the Arduino is missing and the Pi is currently unreachable, so simulator work leads and hardware work is compressed into a bring-up window whenever the hardware returns.

Where the project actually stands

Capability	Status	Evidence
Arduino firmware + protocol	DONE (inherited)	CRC-16 framing, watchdog, fault latching; team's src/ tests pass 8/8
Pi ↔ Arduino driver	WRITTEN, untested on motor	CRC verified against firmware; runs against a simulated Arduino
Lane detection	DONE + validated	10/10 on real track frames; holds from dim 0.55x to bright 1.9x and through heavy rain
Object + stop-sign detection	DONE	~10 FPS on the Pi 5 @ imgsz=320; stop sign = COCO 11
Driving brain (lane/sign/obstacle/U-turn)	DONE + validated	All four behaviours verified in closed-loop simulation, timings exact
Closed-loop simulator	DONE	25 s run, 6.7 m, 100% on track; found 3 real defects
Ultrasonic (HC-SR04) reflex	WRITTEN, needs wiring	Driver loads on the Pi; sensors not yet connected
Driving a physical car	BLOCKED	Arduino physically missing

Three defects were found by the simulator before any hardware testing: an exposure-dependent on-track test, wall-clock timing that made manoeuvres untunable offline, and U-turns chaining into each other. All fixed and verified.

Two decisions taken: the motor layer keeps both implementations for now (the team's tested src/ is untouched), and LANE_LOST deliberately stops and waits rather than searching.

The plan

Wk	Phase	Milestone
1–2	A — Simulator depth	Score the detector over all 300 frames; multi-lane selection (pick MY lane from 3 tape lines)
3–4	A — Control tuning	Tune PD in sim across lighting; record demo runs; stop-sign + obstacle + U-turn demo video
5–6	B — Robustness	Failure-case hunt in sim (glare, dim, lane loss); harden until the sim can't break it
7–8	B — Report + reserve	Write the design report from the measured results; slack for slipped work
9–10	C — Hardware bring-up*	Arduino arrives: serial_check, bench safety tests, teleop, wheels-up verification
11	C — First drives*	Wheels down; lane following on the real track; tune against sim-derived values
12	C — Demo	Ten clean laps; stop sign + obstacle live; final report

* Phases C are **hardware-gated**. If the Arduino arrives earlier, pull them forward and push A/B work later — simulator work is the flexible part. If it never arrives, weeks 9–12 become an extended simulator demo, which is still a complete, defensible deliverable.

Everything here runs on the laptop with no hardware and no network.

Week 1–2 — measure, then improve

- Run `score_detector.py` over the full 300-frame dataset (label them first). One number for the report: MAE and failure rate.
- **Multi-lane selection.** The track has three tape lines; the detector currently locks whichever edges it finds (confidence 0.5 when it sees only one). Decide which pair is 'my lane' and track it between frames.
- Re-score after each change — never tune by eye.

Week 3–4 — tune and demo

- Tune KP/KD in the simulator across the lighting range; record the values and the resulting tracking error.
- Measure U-turn duration in sim to get a starting value for the real car (UTURN_DURATION_S) — it must still be re-measured on the real chassis, but you'll start close.
- Record demo videos: `python -m sim.run_sim --save run.avi` for lane following, stop sign, obstacle and U-turn.

- Adversarial sweep in the simulator: brightness 0.4x to 2.0x, glare up to 1.0, noise up to 0.6, plus lane-loss and off-track starts. Log every configuration that breaks it.
- Fix what breaks; anything you choose not to fix goes in the report as a known limit with the reasoning. (Both current open items — the motor-layer duplication and LANE_LOST stopping dead — are already documented this way.)
- Write the design report from measured numbers, not adjectives. You already have: 10/10 detection, exposure-invariance figures, 100% on-track simulation, ~10 FPS inference, exact behaviour timings, and three defects found pre-hardware.
- Keep `docs/CHANGE_REPORT.md` current — it is the audit trail separating your work from the inherited work.

Compressed because everything upstream is already proven. Do it in this order and do not skip the bench phase.

Step	Command / action	Pass condition
1. Link	<code>python3 tools/serial_check.py</code>	state=DISARMED, no bad_frames
2. Bench	wheels LIFTED	direction correct; E-stop cuts power; watchdog stops motors when USB is p
3. Teleop	<code>python3 -m pi.teleop --term</code>	drive by keyboard, no unexplained faults
4. Ultrasonic	<code>python3 sensors/ultrasonic.py</code>	live distances (mind the ECHO voltage divider)
5. Autonomy	<code>python3 -m control.autodrive --show --record</code>	lane following on the real track
6. Demo	ten consecutive clean laps	with run logs archived

Firmware tweaks to land before wheels-down driving: move PWM off the Timer0 pins (5/6) to Timer1 (9/10), and make the E-stop interrupt-driven with debounce. Both are documented in `docs/HARDWARE_SETUP.md`.

What is deliberately out of scope

Wheel encoders and odometry, IMU fusion, SLAM, and mapping. Encoders are the correct next project: they unlock closed-loop speed and turn the open-loop timed U-turn into a measured one. Attempting SLAM before encoders is the most common way these builds stall.

Biggest risks

Risk	Response
Arduino never arrives	Weeks 9–12 become a simulator demo. Still a complete deliverable — say so early in the report
Simulator success doesn't transfer to the real world	Expect to a degree. The sim gives you starting values, not final ones. Budget week 11 for re-tuning
Multi-lane selection proves hard	Fall back to following a single tape edge at a fixed offset — already works at confidence 0.5.
Time lost to hardware debugging	The bench gate in Phase C exists precisely to stop this. Do not move to wheels-down until every

Next action: label the 300-frame dataset and get a real detector score. Everything else in Phase A builds on that number.